

Você ainda roda INVALIDAÇÃO /* NO CLOUDFRONT?

terminal - cloudfront-invalidation

```
# 'Cache invalidation is one of the 2 hard things' - Phil Karlton
```

```
$ aws cloudfront create-invalidation \
```

```
--distribution-id $CF_ID \
```

```
--paths '/index.html' '/'
```

```
-> Invalidação cirúrgica: 0 downtime, 100% de cache hit nos chunks!
```

Aprenda a sincronizar React + APIs dinâmicas no CI/CD

evitando telas brancas (404), sobrecarga de backend e custos extras.

Por que invalidar /* a cada deploy é um ANTI-PATTERN?

? Purgar Tudo (/*)

- Cache Hit Ratio despenca para ZERO.
- Efeito Manada: Milhares de clientes buscam tudo no S3/Origem de vez.
- Risco de Tela Branca (404) para quem estava navegando durante o build.
- **Ultrapassa os 1.000 caminhos grátis/mês.**

? Invalidação Cirúrgica

- Chunks JS/CSS usam HASH no nome.
- Cache de 1 ano (immutable) na Edge.
- Usuários antigos terminam a sessão nos chunks antigos sem nenhum erro.
- **Invalida SOMENTE o /index.html.**
- **Custo zero e velocidade máxima!**

? REGRA DE OURO: CACHE BUSTING > INVALIDATION

Se o nome do arquivo muda a cada alteração (ex: main.a8f9.js),
ele **NUNCA** precisa ser invalidado no CloudFront.

A Estratégia dos 3 PILARES DE CACHE

Como configurar o Cache-Control correto para cada tipo de arquivo no S3/API:

1. Bundles Hashed (/assets/*.js, *.css)

Cache-Control: `public, max-age=31536000, immutable`

- Fica em cache na Edge por 1 ano. Nunca invalide.

2. Entrypoint SPA (index.html)

Cache-Control: `public, max-age=0, must-revalidate`

- O navegador sempre checa se há nova versão. Invalidação cirúrgica!

3. Backend APIs (/api/cart, /api/catalog)

Cart/Auth: `no-store` | **Catálogo:** `s-maxage=60, stale-while-revalidate=300`

- Garante dados frescos em transações e alta escala no catálogo.

O Pipeline de Deploy Perfeito

GitHub Actions / GitLab CI -> AWS S3 & CloudFront

PASSO 1: Build da Aplicação React / Vite

```
$ npm run build # Gera index.html e /assets/app.[hash].js
```

-> Cria artefatos com hashes exclusivos no diretório dist/

PASSO 2: Sincronizar Chunks PRIMEIRO (Sem index.html)

```
$ aws s3 sync dist/ s3://$BUCKET/ --exclude 'index.html' \
  --cache-control 'public, max-age=31536000, immutable'
```

-> Garante que todos os novos arquivos JS/CSS já estão disponíveis no S3.

PASSO 3 & 4: Subir index.html + Invalidação Cirúrgica

```
$ aws s3 cp dist/index.html s3://$BUCKET/index.html \
  --cache-control 'public, max-age=0, must-revalidate'

$ aws cloudfront create-invalidation --dist-id $CF_ID --paths '/index.html'
```

-> 0 telas brancas, propagação imediata e menor privilégio no IAM!

A Pegadinha da ORDEM DE DEPLOY!

POR QUE A ORDEM IMPORTA NO PIPELINE? ??

Se você subir o index.html ANTES dos novos arquivos JS:

1. O usuário baixa o novo index.html imediatamente.
2. O HTML pede o chunk 'app.new-hash.js' que ainda está subindo.
3. O S3 retorna 404 e o CloudFront pode cachear o erro!
4. Resultado: O cliente vê uma tela branca quebrada.

? SALVE ESTE POST!

Guarde para configurar seus pipelines de CI/CD (GitHub Actions / GitLab)
com a melhor performance e zero downtime na AWS.